CropScan V3 â€” Real-Field Training (clean)

Goal: lift real-world (non-PlantVillage) accuracy from ~50% to 70-80% by attacking the lab-vs-field domain gap.

Approach

1. Train on PlantVillage + PlantDoc (real-world) jointly, with PlantDoc heavily upweighted.
2. Synthesize 'field-style' PlantVillage by segmenting leaves (`rembg`) and pasting them onto PlantDoc background crops.
3. Upgrade backbones: EfficientNetV2-S, MobileNetV3-Large.
4. Strong regularization: RandAugment + MixUp + CutMix + label smoothing.
5. Temperature scaling + abstention thresholds calibrated on real-world data.
6. Report metrics per source so the real-world number isn't averaged away.

Required data on disk (set paths in cell 4):

- PlantVillage 38 class folders (color images)
- PlantDoc with `train/` and `test/` subfolders

Outputs (written to `artifacts/v3/`):

- `efficientnet_v2_s_cropscan_v3.pth` and `mobilenet_v3_large_cropscan_v3.pth`
- matching `*_bundle.pt` files with temperature + thresholds + history
- `training_report.json` , `labels.json`
- preview PNGs

In [2]:
```
import json, math, os, random, subprocess, sys
from collections import Counter, defaultdict
from dataclasses import asdict, dataclass
from io import BytesIO
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from PIL import Image, ImageFilter
from sklearn.metrics import f1_score
from torch.utils.data import DataLoader, Dataset, WeightedRandomSampler
from torchvision import models, transforms

print(f'Python:  {sys.version.split()[0]}')
print(f'PyTorch: {torch.__version__}')
print(f'CUDA:    {torch.cuda.is_available()}')
if torch.cuda.is_available():
    print(f'GPU:     {torch.cuda.get_device_name(0)} ({torch.cuda.get_device_proper
```

```
Python:  3.12.3
PyTorch: 2.6.0+cu124
CUDA:    True
GPU:     NVIDIA RTX A6000 (50.9 GB)
```

In [3]:
```python
SEED = 434
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED); torch.cuda.manual
torch.backends.cudnn.benchmark = True
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')
```

```
Device: cuda
```

Paths and hyperparameters

Edit `PLANTVILLAGE_DIR` and `PLANTDOC_DIR` for your environment. The next cell validates them.

In [4]:
```python
PLANTVILLAGE_DIR = Path('color')
PLANTDOC_DIR     = Path('data/plantdoc')

ARTIFACT_DIR  = Path('artifacts/v3')
SYNTHETIC_DIR = Path('data/synthetic_field')
ARTIFACT_DIR.mkdir(parents=True, exist_ok=True)
SYNTHETIC_DIR.mkdir(parents=True, exist_ok=True)

@dataclass
class Config:
    image_size: int = 224
    batch_size: int = 32
    epochs: int = 25
    warmup_epochs: int = 3
    learning_rate: float = 5e-4
    weight_decay: float = 1e-4
    num_workers: int = 4
    label_smoothing: float = 0.1

    plantvillage_weight: float = 1.0
    synthetic_weight:    float = 2.5
    plantdoc_weight:     float = 6.0

    mixup_alpha: float = 0.2
    cutmix_alpha: float = 1.0
    mix_prob: float = 0.5

    num_synth_per_leaf: int = 2
    max_synth_total: int = 12000
    n_background_patches: int = 2000

    target_confident_precision: float = 0.90

CFG = Config()
print(json.dumps(asdict(CFG), indent=2))
```

```
{
  "image_size": 224,
  "batch_size": 32,
  "epochs": 25,
  "warmup_epochs": 3,
  "learning_rate": 0.0005,
  "weight_decay": 0.0001,
  "num_workers": 4,
  "label_smoothing": 0.1,
  "plantvillage_weight": 1.0,
  "synthetic_weight": 2.5,
  "plantdoc_weight": 6.0,
  "mixup_alpha": 0.2,
  "cutmix_alpha": 1.0,
  "mix_prob": 0.5,
  "num_synth_per_leaf": 2,
  "max_synth_total": 12000,
  "n_background_patches": 2000,
  "target_confident_precision": 0.9
}
```


In [5]:
```
CLASS_NAMES = [
    'Apple___Apple_scab', 'Apple___Black_rot', 'Apple___Cedar_apple_rust', 'Apple__
    'Blueberry___healthy',
    'Cherry_(including_sour)___Powdery_mildew', 'Cherry_(including_sour)___healthy'
    'Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot', 'Corn_(maize)___Common_ru
    'Corn_(maize)___Northern_Leaf_Blight', 'Corn_(maize)___healthy',
    'Grape___Black_rot', 'Grape___Esca_(Black_Measles)',
    'Grape___Leaf_blight_(Isariopsis_Leaf_Spot)', 'Grape___healthy',
    'Orange___Haunglongbing_(Citrus_greening)',
    'Peach___Bacterial_spot', 'Peach___healthy',
    'Pepper__bell___Bacterial_spot', 'Pepper__bell___healthy',
    'Potato___Early_blight', 'Potato___Late_blight', 'Potato___healthy',
    'Raspberry___healthy',
    'Soybean___healthy',
    'Squash___Powdery_mildew',
    'Strawberry___Leaf_scorch', 'Strawberry___healthy',
    'Tomato_Bacterial_spot', 'Tomato_Early_blight', 'Tomato_Late_blight',
    'Tomato_Leaf_Mold', 'Tomato_Septoria_leaf_spot',
    'Tomato_Spider_mites_Two_spotted_spider_mite', 'Tomato__Target_Spot',
    'Tomato__Tomato_YellowLeaf__Curl_Virus', 'Tomato__Tomato_mosaic_virus',
    'Tomato_healthy',
]
CLASS_TO_IDX = {n: i for i, n in enumerate(CLASS_NAMES)}
NUM_CLASSES = len(CLASS_NAMES)
IMG_EXTS = {'.jpg', '.jpeg', '.png', '.webp'}
assert NUM_CLASSES == 38
print(f'{NUM_CLASSES} classes')
```

```
38 classes
```

In [6]:
```
def count_images(root: Path) -> int:
    if not root.exists():
        return 0
    return sum(1 for p in root.rglob('*') if p.suffix.lower() in IMG_EXTS)
```

```
n_pv = count_images(PLANTVILLAGE_DIR)
n_pd = count_images(PLANTDOC_DIR)
print(f'[{"OK " if n_pv else "MISS"}] PlantVillage  {n_pv:>7d} images at {PLANTVILL
print(f'[{"OK " if n_pd else "MISS"}] PlantDoc      {n_pd:>7d} images at {PLANTDOC_

assert n_pv > 0, f'PlantVillage missing at {PLANTVILLAGE_DIR}. Edit PLANTVILLAGE_DI
assert n_pd > 0, f'PlantDoc missing at {PLANTDOC_DIR}. Edit PLANTDOC_DIR.'
```

```
[OK ] PlantVillage    54305 images at color
[OK ] PlantDoc         2579 images at data/plantdoc
```

Class mapping for PlantDoc

PlantDoc folders that don't cleanly correspond to one of the 38 PV classes are dropped (no label noise).

In [7]:
```
PLANTDOC_TO_CROPSCAN = {
    'Apple Scab Leaf': 'Apple___Apple_scab',
    'Apple rust leaf': 'Apple___Cedar_apple_rust',
    'Bell_pepper leaf spot': 'Pepper__bell___Bacterial_spot',
    'Corn Gray leaf spot': 'Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot',
    'Corn leaf blight': 'Corn_(maize)___Northern_Leaf_Blight',
    'Corn rust leaf': 'Corn_(maize)___Common_rust_',
    'Potato leaf early blight': 'Potato___Early_blight',
    'Potato leaf late blight': 'Potato___Late_blight',
    'Squash Powdery mildew leaf': 'Squash___Powdery_mildew',
    'Tomato Early blight leaf': 'Tomato_Early_blight',
    'Tomato Septoria leaf spot': 'Tomato_Septoria_leaf_spot',
    'Tomato leaf bacterial spot': 'Tomato_Bacterial_spot',
    'Tomato leaf late blight': 'Tomato_Late_blight',
    'Tomato leaf mosaic virus': 'Tomato__Tomato_mosaic_virus',
    'Tomato leaf yellow virus': 'Tomato__Tomato_YellowLeaf__Curl_Virus',
    'Tomato mold leaf': 'Tomato_Leaf_Mold',
    'Tomato two spotted spider mites leaf': 'Tomato_Spider_mites_Two_spotted_spider
    'grape leaf black rot': 'Grape___Black_rot',
}
assert set(PLANTDOC_TO_CROPSCAN.values()).issubset(CLASS_TO_IDX)
```

Collect samples

In [8]:
```
@dataclass
class Sample:
    path: Path
    label: int
    source: str

def collect_folder(root: Path, mapping: dict, source: str):
    if not root.exists():
        return []
    out, skipped = [], []
    for folder in sorted(p for p in root.iterdir() if p.is_dir()):
        mapped = mapping.get(folder.name)
        if mapped is None:
```

```
            skipped.append(folder.name)
            continue
        label = CLASS_TO_IDX[mapped]
        for img in folder.rglob('*'):
            if img.suffix.lower() in IMG_EXTS:
                out.append(Sample(img, label, source))
    if skipped:
        print(f'  {root}: skipped {len(skipped)} unmapped folders')
    return out

PV_FOLDER_ALIASES = {
    'Pepper,_bell___Bacterial_spot':                'Pepper__bell___Bacterial_spot
    'Pepper,_bell___healthy':                       'Pepper__bell___healthy',
    'Tomato___Bacterial_spot':                      'Tomato_Bacterial_spot',
    'Tomato___Early_blight':                        'Tomato_Early_blight',
    'Tomato___Late_blight':                         'Tomato_Late_blight',
    'Tomato___Leaf_Mold':                           'Tomato_Leaf_Mold',
    'Tomato___Septoria_leaf_spot':                  'Tomato_Septoria_leaf_spot',
    'Tomato___Spider_mites Two-spotted_spider_mite': 'Tomato_Spider_mites_Two_spott
    'Tomato___Target_Spot':                         'Tomato__Target_Spot',
    'Tomato___Tomato_Yellow_Leaf_Curl_Virus':       'Tomato__Tomato_YellowLeaf__Cu
    'Tomato___Tomato_mosaic_virus':                 'Tomato__Tomato_mosaic_virus',
    'Tomato___healthy':                             'Tomato_healthy',
}
assert set(PV_FOLDER_ALIASES.values()).issubset(CLASS_TO_IDX), 'Alias targets must

pv_map = {n: n for n in CLASS_NAMES}
pv_map.update(PV_FOLDER_ALIASES)
pv_samples = collect_folder(PLANTVILLAGE_DIR, pv_map, 'plantvillage')
print(f'PlantVillage: {len(pv_samples)} images, {len({s.label for s in pv_samples})}
```

```
  color: skipped 1 unmapped folders
PlantVillage: 54305 images, 38/38 classes
```

In [9]:

```
pd_samples = []
for sub in ['train', 'test', '']:
    pd_samples.extend(collect_folder(PLANTDOC_DIR / sub if sub else PLANTDOC_DIR, P
seen = set()
pd_unique = []
for s in pd_samples:
    k = str(s.path.resolve())
    if k not in seen:
        seen.add(k)
        pd_unique.append(s)
pd_samples = pd_unique
print(f'PlantDoc: {len(pd_samples)} images, {len({s.label for s in pd_samples})}/{N
```

```
  data/plantdoc/train: skipped 10 unmapped folders
  data/plantdoc/test: skipped 10 unmapped folders
  data/plantdoc: skipped 3 unmapped folders
PlantDoc: 1728 images, 18/38 classes
```

Stratified 80/10/10 splits per source

In [10]:
```
def stratified_split(samples, val_ratio=0.1, test_ratio=0.1, seed=SEED):
    grouped = defaultdict(list)
    for s in samples:
        grouped[s.label].append(s)
    train, val, test = [], [], []
    rng = random.Random(seed)
    for arr in grouped.values():
        rng.shuffle(arr)
        n = len(arr)
        nt = max(1, int(n * test_ratio)) if n >= 10 else max(0, int(n * test_ratio)
        nv = max(1, int(n * val_ratio))  if n >= 10 else max(0, int(n * val_ratio))
        test.extend(arr[:nt])
        val.extend(arr[nt:nt+nv])
        train.extend(arr[nt+nv:])
    rng.shuffle(train); rng.shuffle(val); rng.shuffle(test)
    return train, val, test

pv_train, pv_val, pv_test = stratified_split(pv_samples, 0.10, 0.10)
pd_train, pd_val, pd_test = stratified_split(pd_samples, 0.15, 0.15)
print(f'pv  train/val/test = {len(pv_train)}/{len(pv_val)}/{len(pv_test)}')
print(f'pd  train/val/test = {len(pd_train)}/{len(pd_val)}/{len(pd_test)}')
```

```
pv  train/val/test = 43471/5417/5417
pd  train/val/test = 1228/250/250
```

Background-randomized synthetic PV

Mine background patches from PlantDoc images, segment PV leaves with `rembg`, paste onto random backgrounds. Cached to disk â€" re-runs are free.

In [11]:
```
def segment_leaf(pil_img, dist_threshold=28, min_fg_ratio=0.05):
    """Segment a PlantVillage leaf from its (near-uniform) background.

    PV images have flat pastel backgrounds, so the median color of the four
    corner patches is a reliable estimate of the background. We mark every
    pixel whose RGB distance from that color exceeds a threshold as foreground.

    Returns an RGBA PIL image (or None if the foreground is too small).
    """
    arr = np.asarray(pil_img.convert('RGB'))
    h, w = arr.shape[:2]
    cs = max(8, min(h, w) // 12)
    corners = np.concatenate([
        arr[:cs, :cs].reshape(-1, 3),
        arr[:cs, -cs:].reshape(-1, 3),
        arr[-cs:, :cs].reshape(-1, 3),
        arr[-cs:, -cs:].reshape(-1, 3),
    ])
    bg = np.median(corners, axis=0).astype(np.float32)
    diff = arr.astype(np.float32) - bg
    dist = np.sqrt((diff ** 2).sum(axis=2))
    mask = (dist > dist_threshold).astype(np.uint8) * 255
    if mask.mean() / 255.0 < min_fg_ratio:
        return None
```

```
    rgba = np.dstack([arr, mask]).astype(np.uint8)
    return Image.fromarray(rgba, 'RGBA')

probe = next((s for s in pv_samples if s.path.exists()), None)
if probe is not None:
    test = segment_leaf(Image.open(probe.path))
    print(f'Segmentation OK on probe: {test is not None and test.size}')
else:
    print('No PV sample to probe')
```

```
Segmentation OK on probe: (256, 256)
```

In [12]:

```
BG_DIR = SYNTHETIC_DIR / 'backgrounds'
BG_DIR.mkdir(parents=True, exist_ok=True)

def build_background_pool(source_samples, n_target, patch_size=384):
    existing = sorted(BG_DIR.glob('bg_*.jpg'))
    if len(existing) >= n_target:
        print(f'  cached background pool: {len(existing)} patches')
        return [str(p) for p in existing[:n_target]]
    rng = random.Random(SEED + 11)
    sources = list(source_samples)
    rng.shuffle(sources)
    saved = list(existing)
    i = len(saved)
    for sample in sources:
        if i >= n_target:
            break
        try:
            img = Image.open(sample.path).convert('RGB')
            w, h = img.size
            if min(w, h) < patch_size:
                scale = patch_size / min(w, h) * 1.1
                img = img.resize((int(w * scale), int(h * scale)))
                w, h = img.size
            for _ in range(2):
                if i >= n_target:
                    break
                x = rng.randint(0, w - patch_size)
                y = rng.randint(0, h - patch_size)
                crop = img.crop((x, y, x + patch_size, y + patch_size))
                crop = crop.filter(ImageFilter.GaussianBlur(radius=3.5))
                out = BG_DIR / f'bg_{i:05d}.jpg'
                crop.save(out, quality=85)
                saved.append(out)
                i += 1
        except Exception:
            continue
    print(f'  saved {len(saved)} background patches')
    return [str(p) for p in saved]

bg_pool = build_background_pool(pd_samples, CFG.n_background_patches)
print(f'Background pool: {len(bg_pool)}')
```

```
  cached background pool: 2000 patches
Background pool: 2000
```

In [13]:

```
SYNTH_DIR = SYNTHETIC_DIR / 'leaves'
SYNTH_DIR.mkdir(parents=True, exist_ok=True)
SYNTH_INDEX = SYNTHETIC_DIR / 'synth_index.json'

def synthesize(pv_samples_, backgrounds, num_per_leaf, max_total):
    if SYNTH_INDEX.exists():
        cached = json.load(open(SYNTH_INDEX))
        recovered = [Sample(Path(e['path']), e['label'], 'synthetic') for e in cach
        if len(recovered) >= 0.9 * max_total:
            print(f'  cached synthetic: {len(recovered)}')
            return recovered
    if not backgrounds:
        print('  no backgrounds available, skipping')
        return []
    rng = random.Random(SEED + 17)
    by_class = defaultdict(list)
    for s in pv_samples_:
        by_class[s.label].append(s)
    leaves_per_class = max(4, (max_total // max(1, len(by_class))) // num_per_leaf)
    print(f'  target: {leaves_per_class} leaves x {num_per_leaf} bgs x {len(by_clas
    out, total = [], 0
    for label, items in by_class.items():
        rng.shuffle(items)
        used = 0
        for source in items:
            if used >= leaves_per_class or total >= max_total:
                break
            try:
                img = Image.open(source.path).convert('RGB')
                cutout = segment_leaf(img)
                if cutout is None:
                    continue
                alpha = np.asarray(cutout)[:, :, 3]
                ys, xs = np.where(alpha > 32)
                if ys.size == 0:
                    continue
                cutout = cutout.crop((int(xs.min()), int(ys.min()), int(xs.max()) +
            except Exception:
                continue
            for variant in range(num_per_leaf):
                if total >= max_total:
                    break
                try:
                    bg = Image.open(rng.choice(backgrounds)).convert('RGB').copy()
                    bw, bh = bg.size
                    target_side = int(min(bw, bh) * rng.uniform(0.55, 0.92))
                    lw, lh = cutout.size
                    scale = target_side / max(lw, lh)
                    leaf = cutout.resize((max(8, int(lw * scale)), max(8, int(lh *
                    leaf = leaf.rotate(rng.uniform(-30, 30), expand=True, resample=
                    lw, lh = leaf.size
                    if lw >= bw or lh >= bh:
                        continue
                    bg.paste(leaf, (rng.randint(0, bw - lw), rng.randint(0, bh - lh
                    out_path = SYNTH_DIR / f'synth_c{label:02d}_{source.path.stem}_
```

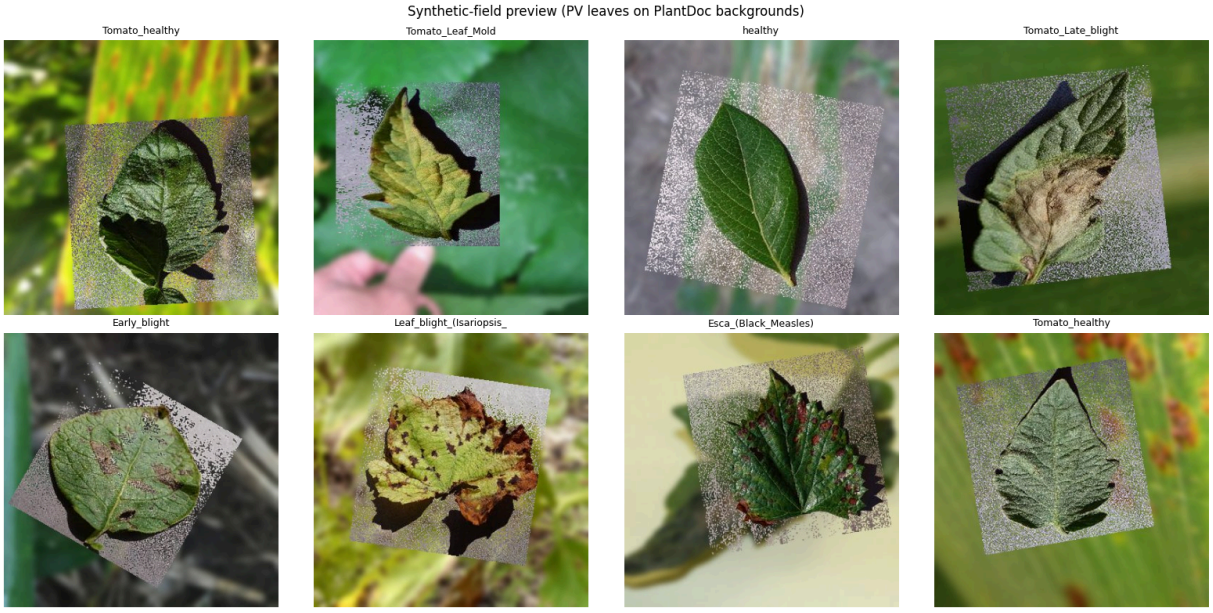
```
                    bg.save(out_path, quality=85)
                    out.append(Sample(out_path, label, 'synthetic'))
                    total += 1
                except Exception:
                    continue
            used += 1
        if total and total % 1000 < num_per_leaf:
            print(f'    progress: {total}/{max_total}')
    json.dump([{'path': str(s.path), 'label': s.label} for s in out], open(SYNTH_IN
    print(f'  generated {len(out)} synthetic samples')
    return out

synth_samples = synthesize(pv_train, bg_pool, CFG.num_synth_per_leaf, CFG.max_synth
print(f'Synthetic-field: {len(synth_samples)}')
```

```
  target: 157 leaves x 2 bgs x 38 classes
  generated 8779 synthetic samples
Synthetic-field: 8779
```

In [14]:

```
import matplotlib.pyplot as plt

if synth_samples:
    rng = random.Random(SEED + 23)
    picks = rng.sample(synth_samples, min(8, len(synth_samples)))
    fig, axes = plt.subplots(2, 4, figsize=(16, 8))
    for ax, s in zip(axes.flat, picks):
        ax.imshow(Image.open(s.path))
        ax.set_title(CLASS_NAMES[s.label].split('___')[-1][:24], fontsize=9)
        ax.axis('off')
    plt.suptitle('Synthetic-field preview (PV leaves on PlantDoc backgrounds)')
    plt.tight_layout()
    plt.savefig(ARTIFACT_DIR / 'synth_preview.png', dpi=100, bbox_inches='tight')
    plt.show()
```

```
/opt/jupyterhub/jupyter_env/lib/python3.12/site-packages/matplotlib/projections/__in
it__.py:63: UserWarning: Unable to import Axes3D. This may be due to multiple versio
ns of Matplotlib being installed (e.g. as a system package and as a pip package). As
a result, the 3D projection is not available.
  warnings.warn("Unable to import Axes3D. This may be due to multiple versions of "
```



Training pool, sampler, transforms

In [15]:
```
train_samples = pv_train + synth_samples + pd_train
val_samples   = pv_val + pd_val
test_sets = {'plantvillage_test': pv_test, 'plantdoc_test': pd_test}
calibration_samples = pd_val if len(pd_val) >= 100 else val_samples

print(f'TRAIN total: {len(train_samples)}  (pv={len(pv_train)}  synth={len(synth_sa
print(f'VAL   total: {len(val_samples)}')
print(f'CALIB total: {len(calibration_samples)}')
for n, s in test_sets.items():
    print(f'TEST  {n:20s} {len(s)}')
```

```
TRAIN total: 53478  (pv=43471  synth=8779  pd=1228)
VAL   total: 5667
CALIB total: 250
TEST  plantvillage_test    5417
TEST  plantdoc_test        250
```

In [16]:
```
SOURCE_WEIGHTS = {
    'plantvillage': CFG.plantvillage_weight,
    'synthetic':    CFG.synthetic_weight,
    'plantdoc':     CFG.plantdoc_weight,
}

def weighted_sampler(samples):
    label_counts = Counter(s.label for s in samples)
    weights = [SOURCE_WEIGHTS.get(s.source, 1.0) / max(1, label_counts[s.label]) fo
    return WeightedRandomSampler(weights, num_samples=len(weights), replacement=Tru
```

In [17]:
```
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(CFG.image_size, scale=(0.5, 1.0), ratio=(0.75, 1.3
```

```
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(p=0.15),
    transforms.RandAugment(num_ops=2, magnitude=9),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    transforms.RandomErasing(p=0.25, scale=(0.02, 0.2)),
])
eval_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(CFG.image_size),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

class LeafDataset(Dataset):
    def __init__(self, samples, tfm):
        self.samples = samples
        self.tfm = tfm
    def __len__(self):
        return len(self.samples)
    def __getitem__(self, i):
        s = self.samples[i]
        try:
            img = Image.open(s.path).convert('RGB')
        except Exception:
            img = Image.new('RGB', (CFG.image_size, CFG.image_size))
        return self.tfm(img), s.label

train_loader = DataLoader(LeafDataset(train_samples, train_transform), batch_size=C
                          sampler=weighted_sampler(train_samples), num_workers=CFG.
                          pin_memory=torch.cuda.is_available(), drop_last=True,
                          persistent_workers=CFG.num_workers > 0)
val_loader = DataLoader(LeafDataset(val_samples, eval_transform), batch_size=CFG.ba
                        shuffle=False, num_workers=CFG.num_workers,
                        pin_memory=torch.cuda.is_available(),
                        persistent_workers=CFG.num_workers > 0)
calibration_loader = DataLoader(LeafDataset(calibration_samples, eval_transform), b
                                shuffle=False, num_workers=CFG.num_workers,
                                pin_memory=torch.cuda.is_available())
test_loaders = {n: DataLoader(LeafDataset(s, eval_transform), batch_size=CFG.batch_
                              shuffle=False, num_workers=CFG.num_workers,
                              pin_memory=torch.cuda.is_available())
                for n, s in test_sets.items() if s}

x, y = next(iter(train_loader))
print(f'Batch: {tuple(x.shape)}  labels [{y.min().item()}, {y.max().item()}]  train
```

```
Batch: (32, 3, 224, 224)  labels [0, 35]  train batches/epoch=1671
```

MixUp + CutMix

In [18]:

```
def mixup(x, y, alpha):
    lam = float(np.random.beta(alpha, alpha)) if alpha > 0 else 1.0
    idx = torch.randperm(x.size(0), device=x.device)
    return lam * x + (1 - lam) * x[idx], y, y[idx], lam
```

```
def cutmix(x, y, alpha):
    lam = float(np.random.beta(alpha, alpha)) if alpha > 0 else 1.0
    idx = torch.randperm(x.size(0), device=x.device)
    _, _, H, W = x.size()
    cr = math.sqrt(1 - lam)
    cw, ch = int(W * cr), int(H * cr)
    cx, cy = int(np.random.randint(W)), int(np.random.randint(H))
    x1, y1 = max(cx - cw // 2, 0), max(cy - ch // 2, 0)
    x2, y2 = min(cx + cw // 2, W), min(cy + ch // 2, H)
    x = x.clone()
    x[:, :, y1:y2, x1:x2] = x[idx, :, y1:y2, x1:x2]
    return x, y, y[idx], 1 - ((x2 - x1) * (y2 - y1) / (W * H))

def mix_loss(crit, logits, ya, yb, lam):
    return lam * crit(logits, ya) + (1 - lam) * crit(logits, yb)
```

Models, scheduler, training

In [19]:
```
def build_model(name):
    if name == 'efficientnet_v2_s':
        m = models.efficientnet_v2_s(weights=models.EfficientNet_V2_S_Weights.IMAGE
        m.classifier = nn.Sequential(nn.Dropout(0.3, inplace=True),
                                     nn.Linear(m.classifier[1].in_features, NUM_CL
        return m
    if name == 'mobilenet_v3_large':
        m = models.mobilenet_v3_large(weights=models.MobileNet_V3_Large_Weights.IMA
        in_f, hid = m.classifier[0].in_features, m.classifier[0].out_features
        m.classifier = nn.Sequential(nn.Linear(in_f, hid), nn.Hardswish(inplace=Tru
                                     nn.Dropout(0.3, inplace=True), nn.Linear(hid,
        return m
    raise ValueError(name)

for n in ['efficientnet_v2_s', 'mobilenet_v3_large']:
    p = sum(x.numel() for x in build_model(n).parameters()) / 1e6
    print(f'{n:25s} {p:.1f}M params')
```

```
efficientnet_v2_s         20.2M params
mobilenet_v3_large        4.3M params
```

In [20]:
```
def warmup_cosine(optimizer, warmup_steps, total_steps, min_ratio=0.01):
    def fn(step):
        if step < warmup_steps:
            return (step + 1) / max(1, warmup_steps)
        prog = (step - warmup_steps) / max(1, total_steps - warmup_steps)
        return max(min_ratio, 0.5 * (1 + math.cos(math.pi * min(1.0, prog))))
    return torch.optim.lr_scheduler.LambdaLR(optimizer, fn)

@torch.no_grad()
def collect_logits(model, loader):
    model.eval()
    L, Y = [], []
    for x, y in loader:
        L.append(model(x.to(device, non_blocking=True)).cpu())
        Y.append(y)
```

```
    return torch.cat(L), torch.cat(Y)

def evaluate_logits(logits, labels, T=1.0):
    p = torch.softmax(logits / T, dim=1)
    pred = p.argmax(1)
    return {
        'accuracy': float((pred == labels).float().mean()),
        'macro_f1': float(f1_score(labels.numpy(), pred.numpy(), average='macro', z
    }
```

In [21]:

```
def train(name):
    save_dir = ARTIFACT_DIR / name
    save_dir.mkdir(parents=True, exist_ok=True)
    model = build_model(name).to(device)
    crit = nn.CrossEntropyLoss(label_smoothing=CFG.label_smoothing)
    opt = torch.optim.AdamW(model.parameters(), lr=CFG.learning_rate, weight_decay=
    total_steps = CFG.epochs * len(train_loader)
    sched = warmup_cosine(opt, CFG.warmup_epochs * len(train_loader), total_steps)
    scaler = torch.amp.GradScaler('cuda', enabled=device.type == 'cuda')
    best_f1, best_state, history = -1.0, None, []
    rng = random.Random(SEED + 41)
    for epoch in range(1, CFG.epochs + 1):
        model.train()
        run_loss, run_n = 0.0, 0
        for x, y in train_loader:
            x = x.to(device, non_blocking=True); y = y.to(device, non_blocking=True
            r = rng.random()
            if r < CFG.mix_prob / 2:
                xm, ya, yb, lam = mixup(x, y, CFG.mixup_alpha); mixed = True
            elif r < CFG.mix_prob:
                xm, ya, yb, lam = cutmix(x, y, CFG.cutmix_alpha); mixed = True
            else:
                xm, ya, yb, lam, mixed = x, y, y, 1.0, False
            opt.zero_grad(set_to_none=True)
            with torch.autocast(device_type=device.type, enabled=device.type == 'cu
                logits = model(xm)
                loss = mix_loss(crit, logits, ya, yb, lam) if mixed else crit(logit
            scaler.scale(loss).backward(); scaler.step(opt); scaler.update(); sched
            run_loss += loss.item() * x.size(0); run_n += x.size(0)
        train_loss = run_loss / max(1, run_n)
        vl, vy = collect_logits(model, val_loader)
        vm = evaluate_logits(vl, vy)
        row = {'epoch': epoch, 'train_loss': round(train_loss, 4),
               'val_acc': round(vm['accuracy'], 4), 'val_f1': round(vm['macro_f1'],
               'lr': round(sched.get_last_lr()[0], 6)}
        history.append(row); print(json.dumps(row))
        torch.save({'state_dict': model.state_dict(), 'epoch': epoch}, save_dir / '
        if vm['macro_f1'] > best_f1:
            best_f1 = vm['macro_f1']
            best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict(
            torch.save({'state_dict': best_state, 'epoch': epoch, 'val_f1': best_f1
    model.load_state_dict(best_state)
    return model, history, best_f1
```

Train EfficientNetV2-S

In [22]:
```
effnet_model, effnet_history, effnet_best = train('efficientnet_v2_s')
print(f'EfficientNetV2-S best val_f1 = {effnet_best:.4f}')
```

```
{"epoch": 1, "train_loss": 1.8684, "val_acc": 0.9635, "val_f1": 0.9524, "lr": 0.0001
67}
{"epoch": 2, "train_loss": 1.1721, "val_acc": 0.9647, "val_f1": 0.9587, "lr": 0.0003
33}
{"epoch": 3, "train_loss": 1.1526, "val_acc": 0.9677, "val_f1": 0.9591, "lr": 0.000
5}
{"epoch": 4, "train_loss": 1.172, "val_acc": 0.9718, "val_f1": 0.9584, "lr": 0.00049
7}
{"epoch": 5, "train_loss": 1.1266, "val_acc": 0.9562, "val_f1": 0.9459, "lr": 0.0004
9}
{"epoch": 6, "train_loss": 1.0855, "val_acc": 0.9661, "val_f1": 0.9581, "lr": 0.0004
77}
{"epoch": 7, "train_loss": 1.0649, "val_acc": 0.97, "val_f1": 0.9617, "lr": 0.00046}
{"epoch": 8, "train_loss": 1.0595, "val_acc": 0.9765, "val_f1": 0.9699, "lr": 0.0004
39}
{"epoch": 9, "train_loss": 1.0284, "val_acc": 0.9728, "val_f1": 0.9642, "lr": 0.0004
14}
{"epoch": 10, "train_loss": 1.0149, "val_acc": 0.9767, "val_f1": 0.9697, "lr": 0.000
385}
{"epoch": 11, "train_loss": 0.9993, "val_acc": 0.9756, "val_f1": 0.9699, "lr": 0.000
354}
{"epoch": 12, "train_loss": 0.9985, "val_acc": 0.9794, "val_f1": 0.9737, "lr": 0.000
32}
{"epoch": 13, "train_loss": 0.9763, "val_acc": 0.9771, "val_f1": 0.9721, "lr": 0.000
286}
{"epoch": 14, "train_loss": 0.957, "val_acc": 0.9792, "val_f1": 0.9755, "lr": 0.0002
5}
{"epoch": 15, "train_loss": 0.9482, "val_acc": 0.9806, "val_f1": 0.9755, "lr": 0.000
214}
{"epoch": 16, "train_loss": 0.9506, "val_acc": 0.9829, "val_f1": 0.9765, "lr": 0.000
18}
{"epoch": 17, "train_loss": 0.9388, "val_acc": 0.9827, "val_f1": 0.9769, "lr": 0.000
146}
{"epoch": 18, "train_loss": 0.9384, "val_acc": 0.9827, "val_f1": 0.9753, "lr": 0.000
115}
{"epoch": 19, "train_loss": 0.9288, "val_acc": 0.9838, "val_f1": 0.9772, "lr": 8.6e-
05}
{"epoch": 20, "train_loss": 0.9135, "val_acc": 0.9838, "val_f1": 0.9783, "lr": 6.1e-
05}
{"epoch": 21, "train_loss": 0.9038, "val_acc": 0.9838, "val_f1": 0.978, "lr": 4e-05}
{"epoch": 22, "train_loss": 0.9152, "val_acc": 0.9843, "val_f1": 0.9784, "lr": 2.3e-
05}
{"epoch": 23, "train_loss": 0.9115, "val_acc": 0.9838, "val_f1": 0.9779, "lr": 1e-0
5}
{"epoch": 24, "train_loss": 0.912, "val_acc": 0.9839, "val_f1": 0.9783, "lr": 5e-06}
{"epoch": 25, "train_loss": 0.9044, "val_acc": 0.9841, "val_f1": 0.9785, "lr": 5e-0
6}
EfficientNetV2-S best val_f1 = 0.9785
```

Train MobileNetV3-Large

In [23]:
```
mobnet_model, mobnet_history, mobnet_best = train('mobilenet_v3_large')
print(f'MobileNetV3-Large best val_f1 = {mobnet_best:.4f}')
```

```
{"epoch": 1, "train_loss": 1.9318, "val_acc": 0.9531, "val_f1": 0.9421, "lr": 0.0001
67}
{"epoch": 2, "train_loss": 1.2326, "val_acc": 0.9585, "val_f1": 0.9506, "lr": 0.0003
33}
{"epoch": 3, "train_loss": 1.1736, "val_acc": 0.9592, "val_f1": 0.9462, "lr": 0.000
5}
{"epoch": 4, "train_loss": 1.1359, "val_acc": 0.9675, "val_f1": 0.9622, "lr": 0.0004
97}
{"epoch": 5, "train_loss": 1.1174, "val_acc": 0.9612, "val_f1": 0.9552, "lr": 0.0004
9}
{"epoch": 6, "train_loss": 1.0762, "val_acc": 0.9606, "val_f1": 0.9501, "lr": 0.0004
77}
{"epoch": 7, "train_loss": 1.0384, "val_acc": 0.9721, "val_f1": 0.9675, "lr": 0.0004
6}
{"epoch": 8, "train_loss": 1.0461, "val_acc": 0.9714, "val_f1": 0.9659, "lr": 0.0004
39}
{"epoch": 9, "train_loss": 1.0135, "val_acc": 0.9751, "val_f1": 0.9678, "lr": 0.0004
14}
{"epoch": 10, "train_loss": 1.0049, "val_acc": 0.9748, "val_f1": 0.9694, "lr": 0.000
385}
{"epoch": 11, "train_loss": 1.0027, "val_acc": 0.9769, "val_f1": 0.9688, "lr": 0.000
354}
{"epoch": 12, "train_loss": 0.9963, "val_acc": 0.9751, "val_f1": 0.9687, "lr": 0.000
32}
{"epoch": 13, "train_loss": 0.9846, "val_acc": 0.9804, "val_f1": 0.9745, "lr": 0.000
286}
{"epoch": 14, "train_loss": 0.9502, "val_acc": 0.9786, "val_f1": 0.9714, "lr": 0.000
25}
{"epoch": 15, "train_loss": 0.9631, "val_acc": 0.976, "val_f1": 0.9698, "lr": 0.0002
14}
{"epoch": 16, "train_loss": 0.954, "val_acc": 0.9804, "val_f1": 0.9727, "lr": 0.0001
8}
{"epoch": 17, "train_loss": 0.9564, "val_acc": 0.9806, "val_f1": 0.9746, "lr": 0.000
146}
{"epoch": 18, "train_loss": 0.9461, "val_acc": 0.9804, "val_f1": 0.9729, "lr": 0.000
115}
{"epoch": 19, "train_loss": 0.9313, "val_acc": 0.9799, "val_f1": 0.9726, "lr": 8.6e-
05}
{"epoch": 20, "train_loss": 0.9299, "val_acc": 0.9813, "val_f1": 0.9756, "lr": 6.1e-
05}
{"epoch": 21, "train_loss": 0.9298, "val_acc": 0.982, "val_f1": 0.9759, "lr": 4e-05}
{"epoch": 22, "train_loss": 0.9339, "val_acc": 0.9824, "val_f1": 0.9761, "lr": 2.3e-
05}
{"epoch": 23, "train_loss": 0.9177, "val_acc": 0.9824, "val_f1": 0.976, "lr": 1e-05}
{"epoch": 24, "train_loss": 0.9351, "val_acc": 0.9822, "val_f1": 0.976, "lr": 5e-06}
{"epoch": 25, "train_loss": 0.9314, "val_acc": 0.9822, "val_f1": 0.9761, "lr": 5e-0
6}
MobileNetV3-Large best val_f1 = 0.9761
```

Calibration on real-world held-out (PlantDoc val)

In [24]:

```python
def fit_temperature(logits, labels):
    lo, la = logits.to(device), labels.to(device)
    t = torch.ones(1, device=device, requires_grad=True)
    opt = torch.optim.LBFGS([t], lr=0.01, max_iter=50)
    crit = nn.CrossEntropyLoss()
    def closure():
        opt.zero_grad()
        loss = crit(lo / t.clamp(0.5, 5.0), la)
        loss.backward()
        return loss
    opt.step(closure)
    return float(t.detach().clamp(0.5, 5.0).item())

def uncertainty(logits, T):
    p = torch.softmax(logits / T, dim=1)
    top2 = p.topk(2, dim=1).values
    return (p.argmax(1), top2[:, 0], top2[:, 0] - top2[:, 1],
            -(p * torch.log(p.clamp_min(1e-8))).sum(1) / math.log(NUM_CLASSES))

def search_thresholds(logits, labels, T):
    pred, mp, mg, en = uncertainty(logits, T)
    correct = pred == labels
    best = None
    for c in np.linspace(0.45, 0.95, 11):
        for m in np.linspace(0.05, 0.45, 9):
            for e in np.linspace(0.25, 0.85, 13):
                mask = (mp >= c) & (mg >= m) & (en <= e)
                if mask.sum() < 20:
                    continue
                prec = float(correct[mask].float().mean())
                cov = float(mask.float().mean())
                if prec >= CFG.target_confident_precision and (best is None or cov
                    best = {'max_prob_min': round(float(c), 4), 'margin_min': round
                            'entropy_max': round(float(e), 4),
                            'confident_precision': round(prec, 4), 'confident_cover
    return best or {'max_prob_min': 0.70, 'margin_min': 0.20, 'entropy_max': 0.55,
                    'confident_precision': None, 'confident_coverage': None}

def calibrate(model, name):
    vl, vy = collect_logits(model, val_loader)
    cl, cy = collect_logits(model, calibration_loader)
    T = fit_temperature(vl, vy)
    thr = search_thresholds(cl, cy, T)
    print(f'{name}: T={T:.3f}  thresholds={thr}')
    return T, thr

effnet_T, effnet_thr = calibrate(effnet_model, 'efficientnet_v2_s')
mobnet_T, mobnet_thr = calibrate(mobnet_model, 'mobilenet_v3_large')
```

```
efficientnet_v2_s: T=0.887  thresholds={'max_prob_min': 0.7, 'margin_min': 0.2, 'ent
ropy_max': 0.55, 'confident_precision': None, 'confident_coverage': None}
mobilenet_v3_large: T=0.881  thresholds={'max_prob_min': 0.7, 'margin_min': 0.2, 'en
tropy_max': 0.55, 'confident_precision': None, 'confident_coverage': None}
```

Per-source evaluation (the honesty cell)

In [25]:
```python
def evaluate_all(model, T, name):
    print(f'\n=== {name} (T={T:.3f}) ===')
    out = {}
    for src, ldr in test_loaders.items():
        lg, lb = collect_logits(model, ldr)
        m = evaluate_logits(lg, lb, T)
        m['n'] = int(lb.size(0))
        out[src] = m
        print(f'  {src:25s} n={m["n"]:>5d}  acc={m["accuracy"]:.4f}  f1={m["macro_f
    return out

effnet_per_source = evaluate_all(effnet_model, effnet_T, 'efficientnet_v2_s')
mobnet_per_source = evaluate_all(mobnet_model, mobnet_T, 'mobilenet_v3_large')
```

```
=== efficientnet_v2_s (T=0.887) ===
  plantvillage_test         n= 5417  acc=0.9976  f1=0.9961
  plantdoc_test             n=  250  acc=0.6920  f1=0.6115

=== mobilenet_v3_large (T=0.881) ===
  plantvillage_test         n= 5417  acc=0.9972  f1=0.9959
  plantdoc_test             n=  250  acc=0.6680  f1=0.6388
```

Test-time augmentation (5-crop + flips)

In [26]:
```python
tta_base = transforms.Compose([
    transforms.Resize(256), transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

def five_crop_flips(t, sz):
    _, h, w = t.shape
    crops = []
    cy, cx = (h - sz) // 2, (w - sz) // 2
    for (y, x) in [(0, 0), (0, w - sz), (h - sz, 0), (h - sz, w - sz), (cy, cx)]:
        c = t[:, y:y+sz, x:x+sz]
        crops.append(c)
        crops.append(torch.flip(c, dims=[2]))
    return torch.stack(crops)

@torch.no_grad()
def predict_tta(model, image_path, T):
    img = Image.open(image_path).convert('RGB')
    crops = five_crop_flips(tta_base(img), CFG.image_size).to(device)
    model.eval()
    return torch.softmax(model(crops) / T, dim=1).mean(0).cpu()

def evaluate_tta(model, samples, T, max_n=400):
```

```
    rng = random.Random(SEED + 71)
    sub = samples if len(samples) <= max_n else rng.sample(samples, max_n)
    correct = sum(1 for s in sub if int(predict_tta(model, s.path, T).argmax()) ==
    return correct / max(1, len(sub)), len(sub)

for (m, T, name) in [(effnet_model, effnet_T, 'efficientnet_v2_s'),
                     (mobnet_model, mobnet_T, 'mobilenet_v3_large')]:
    print(f'\n--- TTA: {name} ---')
    for tn, ts in test_sets.items():
        if ts:
            acc, n = evaluate_tta(m, ts, T)
            print(f'  {tn:25s} TTA acc={acc:.4f} (n={n})')
```

```
--- TTA: efficientnet_v2_s ---
  plantvillage_test         TTA acc=1.0000 (n=400)
  plantdoc_test             TTA acc=0.6840 (n=250)

--- TTA: mobilenet_v3_large ---
  plantvillage_test         TTA acc=1.0000 (n=400)
  plantdoc_test             TTA acc=0.6680 (n=250)
```

Save bundles + report

In [27]:

```
def save_bundle(name, model, history, T, thr, per_source):
    w = ARTIFACT_DIR / f'{name}_cropscan_v3.pth'
    b = ARTIFACT_DIR / f'{name}_cropscan_v3_bundle.pt'
    torch.save(model.state_dict(), w)
    torch.save({
        'state_dict': model.state_dict(),
        'temperature': T,
        'thresholds': thr,
        'class_names': CLASS_NAMES,
        'version': 'cropscan-v3',
        'architecture': name,
        'history': history,
        'per_source_test_metrics': per_source,
    }, b)
    print(f'  {w}')
    print(f'  {b}')

save_bundle('efficientnet_v2_s',  effnet_model, effnet_history, effnet_T, effnet_th
save_bundle('mobilenet_v3_large', mobnet_model, mobnet_history, mobnet_T, mobnet_th

report = {
    'class_names': CLASS_NAMES,
    'data_sizes': {
        'train_total': len(train_samples),
        'pv_train': len(pv_train), 'synthetic': len(synth_samples), 'pd_train': len
        'val_total': len(val_samples), 'calibration_total': len(calibration_samples
    },
    'test_sizes': {k: len(v) for k, v in test_sets.items()},
    'efficientnet_v2_s': {'history': effnet_history, 'temperature': effnet_T,
                          'thresholds': effnet_thr, 'per_source': effnet_per_source
    'mobilenet_v3_large': {'history': mobnet_history, 'temperature': mobnet_T,
                           'thresholds': mobnet_thr, 'per_source': mobnet_per_sour
```

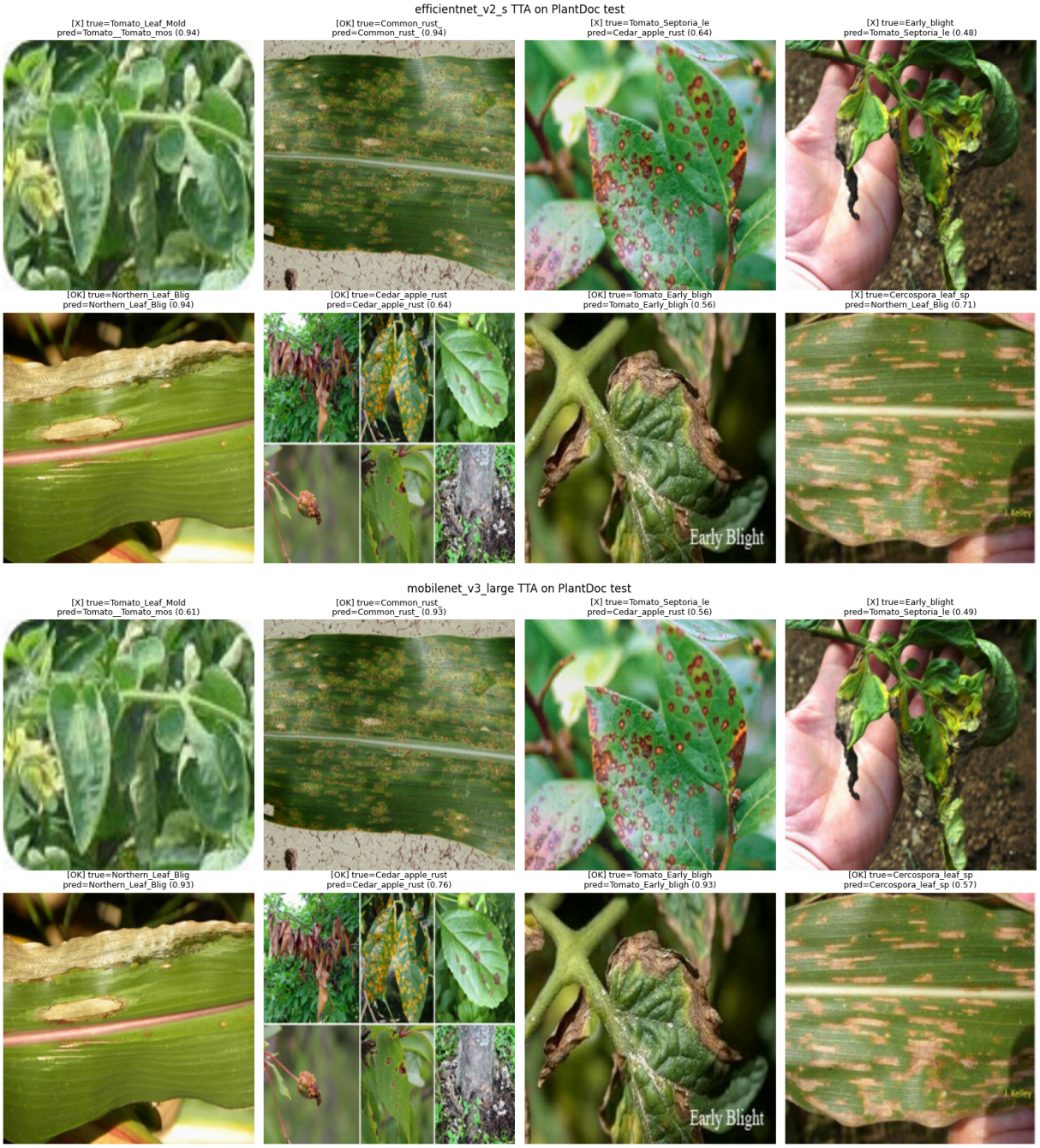
```
}
json.dump(report, open(ARTIFACT_DIR / 'training_report.json', 'w'), indent=2)
json.dump(CLASS_NAMES, open(ARTIFACT_DIR / 'labels.json', 'w'), indent=2)
print(f'\nReport: {ARTIFACT_DIR / "training_report.json"}')
```

```
 artifacts/v3/efficientnet_v2_s_cropscan_v3.pth
 artifacts/v3/efficientnet_v2_s_cropscan_v3_bundle.pt
 artifacts/v3/mobilenet_v3_large_cropscan_v3.pth
 artifacts/v3/mobilenet_v3_large_cropscan_v3_bundle.pt

Report: artifacts/v3/training_report.json
```

Visualize TTA predictions on PlantDoc test

In [28]:

```
def visualize(model, samples, name, T, n=8):
    if not samples:
        return
    rng = random.Random(SEED + 101)
    picks = rng.sample(samples, min(n, len(samples)))
    fig, axes = plt.subplots(2, 4, figsize=(16, 9))
    for ax, s in zip(axes.flat, picks):
        probs = predict_tta(model, s.path, T)
        top = int(probs.argmax())
        ax.imshow(Image.open(s.path).convert('RGB').resize((224, 224)))
        ax.axis('off')
        mark = 'OK' if top == s.label else 'X'
        true = CLASS_NAMES[s.label].split('___')[-1][:18]
        pred = CLASS_NAMES[top].split('___')[-1][:18]
        ax.set_title(f'[{mark}] true={true}\npred={pred} ({probs[top]:.2f})', fonts
    plt.suptitle(f'{name} TTA on PlantDoc test')
    plt.tight_layout()
    plt.savefig(ARTIFACT_DIR / f'preview_{name}.png', dpi=100, bbox_inches='tight')
    plt.show()

visualize(effnet_model, pd_test, 'efficientnet_v2_s', effnet_T)
visualize(mobnet_model, pd_test, 'mobilenet_v3_large', mobnet_T)
```



Backend integration

v3 uses EfficientNetV2-S + MobileNetV3-Large (not B0/V2). To deploy:

1. Copy into `backend/models/`:
   - `efficientnet_v2_s_cropscan_v3.pth`
   - `mobilenet_v3_large_cropscan_v3.pth`
   - `efficientnet_v2_s_cropscan_v3_bundle.pt`
   - `mobilenet_v3_large_cropscan_v3_bundle.pt`
   - `training_report.json`, `labels.json`
2. Replace builders in `backend/app/inference.py`:

```
def _build_efficientnet_v2_s() -> nn.Module:
    m = models.efficientnet_v2_s(weights=None)
    m.classifier = nn.Sequential(
        nn.Dropout(0.3, inplace=True),
        nn.Linear(m.classifier[1].in_features, len(CLASS_NAMES)),
    )
    return m

def _build_mobilenet_v3_large() -> nn.Module:
    m = models.mobilenet_v3_large(weights=None)
    in_f, hid = m.classifier[0].in_features, m.classifier[0].out_features
    m.classifier = nn.Sequential(
        nn.Linear(in_f, hid), nn.Hardswish(inplace=True),
        nn.Dropout(0.3, inplace=True), nn.Linear(hid, len(CLASS_NAMES)),
    )
    return m
```

3. Update the model registry and file paths in `inference.py`. Wire the calibrated `temperature` and `thresholds` from `training_report.json` into the existing abstention logic.

In [ ]: